

Perceptron et reconnaissance d'écriture manuscrite

Rapport de projet

Raad Eloi 21964085
L3 Physique, Université de Paris
Décembre 2022



**Université
de Paris**

1. Introduction	2
2. Algorithme du Perceptron	2
2.1. Perceptron de Rosenblatt	3
2.2. Règle d'apprentissage du perceptron	5
2.3. Multiclass perceptron	5
3. Méthodologie	7
3.1. Le MNIST	7
3.2. Perceptron monocouche	8
3.3. Perceptron multicouche	8
3.4. Entraînement	9
3.5. Evaluation des performances	9
4. Résultats	10
4.1. Perceptron monocouche	10
4.2. Perceptron Multicouche	12
5. Conclusion	15
6. Références	16

1. Introduction

Ce rapport porte sur le projet Perceptron et reconnaissance d'écriture manuscrite dans le cadre de l'UE Physique Numérique. Le but du projet est d'implémenter l'algorithme du Perceptron (classique et multiclass) et d'étudier les performances en fonction des différents paramètres du système.

2. Algorithme du Perceptron

Le Perceptron est un algorithme de classification binaire par apprentissage supervisé. C'est un neurone formel, il classe linéairement les données grâce à une règle d'apprentissage permettant de déterminer les poids des synapses.

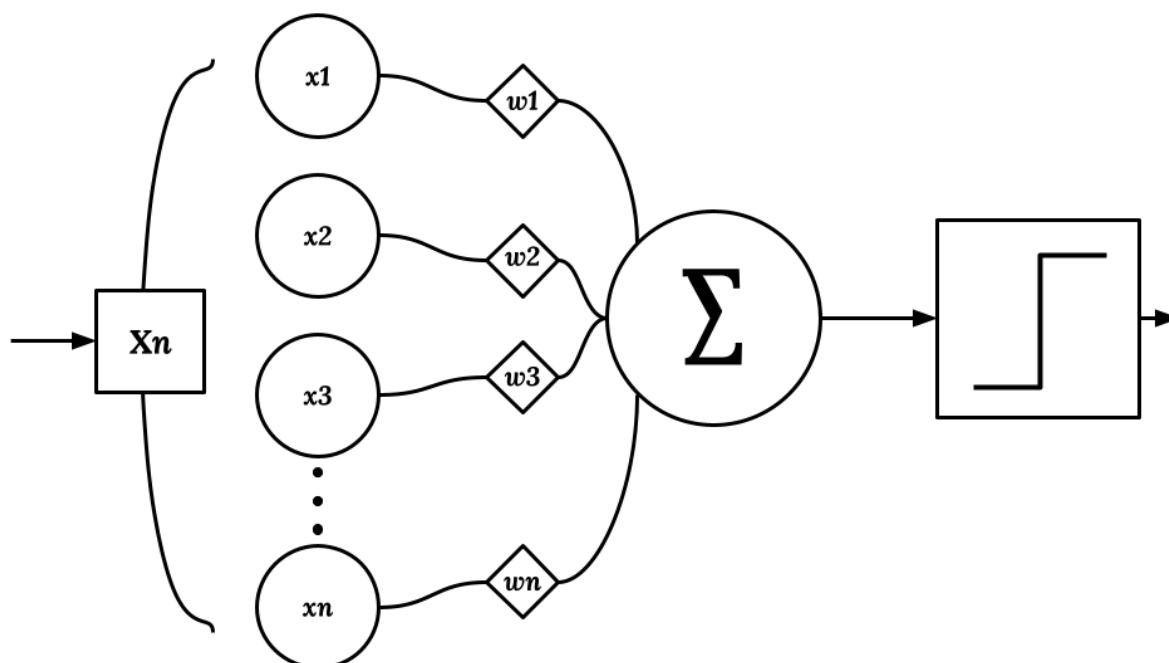


Figure 1 : Schéma d'un neurone formel

On voit qu'un jeu de données est donné en entrée auquel on applique des poids et le tout est sommé ; on passe ensuite par la fonction d'activation qui transforme le résultat de la somme précédente. Il existe plusieurs fonctions d'activations dont nous étudierons un nombre non exhaustif.

2.1. Perceptron de Rosenblatt

Frank Rosenblatt inventa en 1957 l'algorithme suivant :

```
Input : jeu de données  $\{(X_1, \sigma_1^*), \dots, (X_n, \sigma_n^*)\}$ ,  
1 Initialiser  $w = 0$   
2 Répéter  
3   Pour  $i = 1$  à  $n$   
4       Si  $\sigma_i^*(w \cdot X_i) \leq 0$  alors  
5            $w \leftarrow w + \sigma_i^* X_i$   
6   Fin pour  
7 jusqu'à ce qu'il n'y ait plus d'erreurs  
8 Retourner  $w$ 
```

X_n est un vecteur tel que $X_n = (x_1^n, \dots, x_{m+1}^n) \in \mathbb{R}^{m+1}$ et $\sigma_n \in \{-1, 1\}$, le vecteur X_n est alors un exemple, le point σ_n sa classe donnée par le perceptron et σ_n^* sa classe réelle. Le vecteur w quant à lui représente les poids attribués à chacune des composantes de X_n ; α est learning rate (ou taux d'apprentissage) un réel entre 0 et 1.

Cet algorithme est un cas particulier de l'algorithme de la descente de gradient avec comme fonction objectif $f(w) = \sum_{i \in \xi} \sigma_i^*(w \cdot X_i)$ avec ξ l'ensemble des exemples mal classifiés. L'algorithme converge si et seulement si les exemples sont linéairement séparables et cela en au plus $(\frac{R}{\gamma})^2$.

Procédons à un raisonnement par récurrence :

Notons w_k les valeurs de w lors de l'exécution de l'algorithme.

Supposons que l'algorithme fait k erreurs à l'élément X_t .

On note w^* le vecteur normalisé tel que les exemples soient parfaitement classifiés.

w^* forme un hyperplan de l'espace.

$$\sigma_t^*(X_t \cdot w^*) \geq \gamma$$

En exécutant l'algorithme on à :

$$\begin{aligned} (w_{k+1} \cdot w^*) &= ([w_k + \sigma_t^*] \cdot w^*) = (w_k \cdot w^*) + \sigma_t^*(X_t \cdot w^*) \\ (w_{k+1} \cdot w^*) &\geq (w_k \cdot w^*) + \gamma \end{aligned}$$

À l'élément suivant :

$$(w_{k+2} \cdot w^*) \geq (w_{k+1} \cdot w^*) + \gamma \geq (w_k \cdot w^*) + 2\gamma$$

Donc :

$$(w_{k+2} \cdot w^*) \geq (w_k \cdot w^*) + 2\gamma$$

À $k = 1$:

$$\begin{aligned} (w_2 \cdot w^*) &\geq (w_1 \cdot w^*) + \gamma = \gamma \text{ puisque } w_1 = 0 \\ (w_3 \cdot w^*) &\geq (w_2 \cdot w^*) + \gamma \geq 2\gamma \end{aligned}$$

On a donc :

$$(w_{k+1} \cdot w^*) \geq k\gamma$$

L'inégalité de Cauchy nous dit que $\|w_{k+1}\| \|w^*\| \geq (w_{k+1} \cdot w^*)$ or $\|w^*\| = 1$.

Donc :

$$\|w_{k+1}\| \geq k\gamma$$

Aussi :

$$\begin{aligned} \|w_{k+1}\|^2 &= \|w_k + \sigma_t^* X_t\|^2 \\ \|w_{k+1}\|^2 &= \|w_k\|^2 + \sigma_t^{*2} \|X_t\|^2 + 2\sigma_t^*(X_t \cdot w_k) \end{aligned}$$

On pose : $\|X_t\| \leq R$

$$(X_t \cdot w_k) \leq 0 \text{ et } \|X_t\|^2 = \sigma_t^{*2} \|X_t\|^2 \leq R^2$$

Alors :

$$\begin{aligned} \|w_{k+1}\|^2 &\leq \|w_k\|^2 + R^2 \\ \|w_{k+2}\|^2 &\leq \|w_{k+1}\|^2 + R^2 \leq 2R^2 \end{aligned}$$

À $k = 1$:

$$\begin{aligned} \|w_2\|^2 &\leq \|w_1\|^2 + R^2 = R^2 \\ \|w_3\|^2 &\leq \|w_2\|^2 + R^2 \leq 2R^2 \end{aligned}$$

Donc :

$$\|w_{k+1}\|^2 \leq kR^2$$

On a alors la $k^{\text{ème}}$ erreur tel que :

$$k \leq \frac{R^2}{\gamma^2}$$

L'algorithme du Perceptron procède donc en quatre temps :

- Initialisation des paramètres (les poids ici)
- Propagation avant, on applique w_k à X_i
- Correction de w , si il y a une erreur
- Propagation avant avec w_{k+1} et X_i

En fonction des données d'entrée le nombre d'erreurs peut être très important on peut limiter le nombre d'itérations de l'algorithme.

2.2. Règle d'apprentissage du perceptron

Le perceptron de Rosenblatt utilise la règle d'apprentissage suivante, $w_n' = w_n + \alpha(\sigma_n^* \cdot X_n)$ qui ne prend en compte que la classe donnée par la fonction d'activation signum, ainsi tous les poids sont corrigés jusqu'à convergence.

Une autre approche consiste en l'utilisation de la règle suivante, $w_n' = w_n + \alpha(\sigma_n^* - \sigma_n) \cdot X_n$ ainsi on peut ne pas modifier les poids qui mènent au bon résultat tout en continuant de modifier les autres. Si $\sigma_n > 0$ et $\sigma_n^* < 0$ alors le poids diminue, dans le cas opposé il augmente et si les deux valeurs sont du même signe alors il ne bouge pas. Cette approche permet aussi d'utiliser d'autres fonctions d'activation (step, ReLu ou encore tanh).

2.3. Multiclass perceptron

Le perceptron multicouche permet (comme son nom anglais l'indique) de traiter des cas de classifications de données à multiple classes. Il consiste en un minimum de trois couches :

- Une couche d'entrée où l'on applique les poids d'entrée
- Une couche intermédiaire (ou cachée), les valeurs précédentes passent dans une première fonction d'activation (ReLu, eLu, tanh, sigmoid, etc)
- Une de sortie, on applique des poids aux valeurs de la couche intermédiaire puis on applique une autre fonction d'activation (une fonction comme softmax permet d'avoir des probabilités)

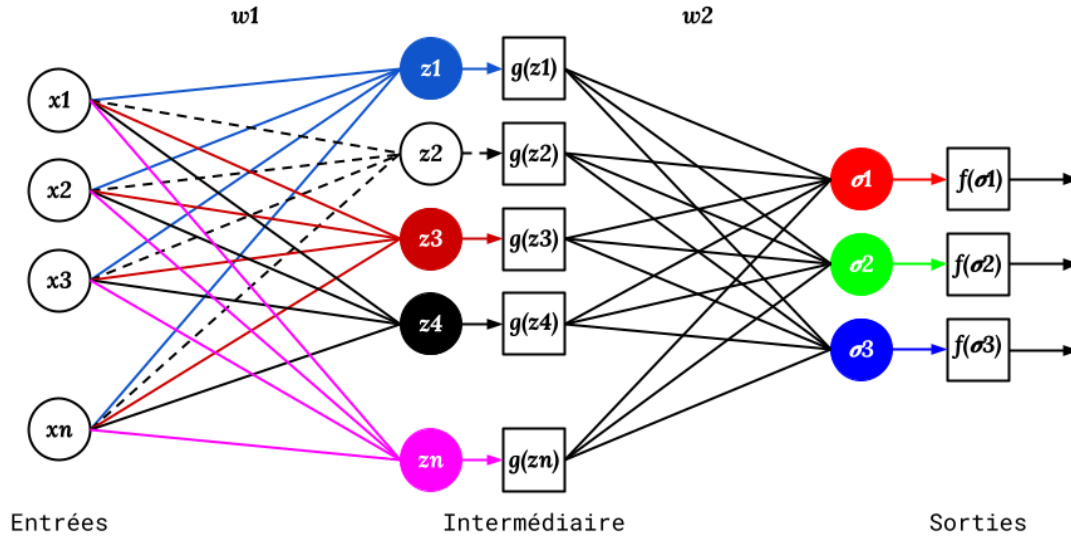


Figure 2 : Schéma d'un réseau de neurones

Le perceptron monocouche étant considéré comme un neurone, le perceptron multicouche devient alors un réseau de neurones. La règle (ou théorie) de Hebb nous dit que : Dans un réseau de neurones artificiels lorsque deux neurones sont excités conjointement, ils créent ou renforcent un lien les unissant. Ainsi les poids sont liés et la modification d'un poids impactera tous le système il faut donc établir comment la modification aussi petite soit-elle d'un poids impacte les reste des poids. Tout comme pour l'algorithme du perceptron monocouche on va faire une descente de gradient, la fonction que l'on cherche à minimiser est appelée "loss" il en existe plusieurs comme "categorical crossentropy" mais plus simplement on peut utiliser la variance, $Var = (\sigma_n - \sigma_n^*)^2$. Le gradient de la variance est alors composé des dérivées partielles de Var par rapport à tous les différents poids et biais du système.

Dans le cas de figure où l'on a un unique neurone par couche (et n couches), on aura :

$$\begin{aligned} \frac{dVar}{dw_n} &= \frac{d\sigma_n}{dw_n} \frac{dVar}{d\sigma_n} = \frac{dz_n}{dw_n} \frac{d\sigma_n}{dz_n} \frac{dVar}{d\sigma_n} \\ \frac{dVar}{db_n} &= \frac{dz_n}{db_n} \frac{d\sigma_n}{dz_n} \frac{dVar}{d\sigma_n} \\ \frac{dVar}{d\sigma_{n-1}} &= \frac{dz_n}{d\sigma_{n-1}} \frac{d\sigma_n}{dz_n} \frac{dVar}{d\sigma_n} \end{aligned}$$

$$\sigma_n = g(z_n)$$

$$z_n = w_n \sigma_{n-1} + b_n$$

On a donc :

$$\frac{dVar}{dw_n} = \sigma_{n-1} g'(z_n) 2(\sigma_n - \sigma_n^*)$$

$$\frac{dVar}{db_n} = g'(z_n) 2(\sigma_n - \sigma_n^*)$$

Dans le cas ou chaque couche possède plusieurs neurones (la couche n à j neurones et la couche n-1, k neurones) ces équations deviennent :

$$\frac{dVar}{dw_n^{jk}} = \sigma_{n-1}^k g'(z_n^j) \frac{dVar}{d\sigma_n^j}$$

$$\frac{dVar}{d\sigma_n^j} = \sum_{k=0}^{n-1} w_{n+1}^{jk} g'(z_{n+1}^j) \frac{dVar}{d\sigma_{n+1}^j} = 2(\sigma_n^j - \sigma_n^{j*})$$

3. Méthodologie

3.1. Le MNIST

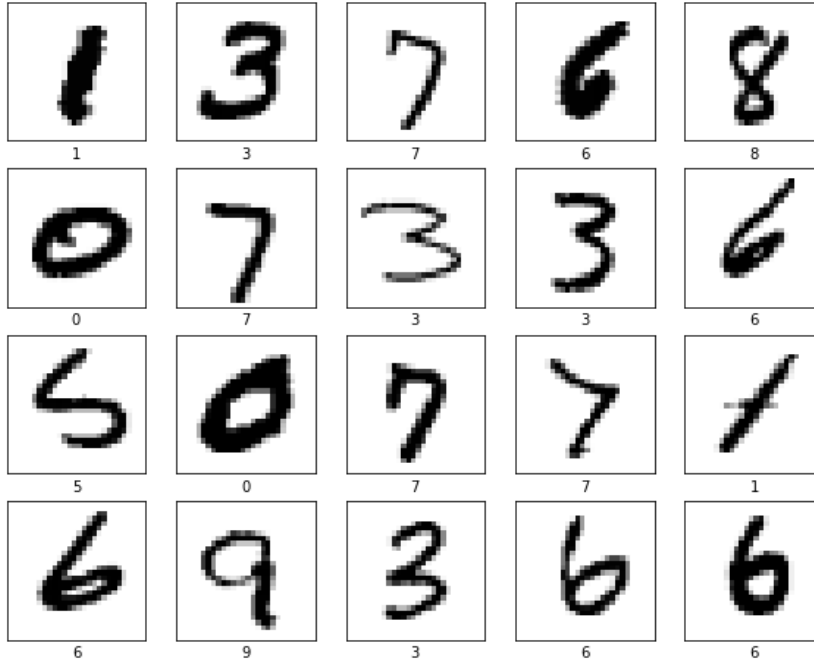


Figure 3 : Images du MNIST avec leurs label

Le MNIST est une base de 70000 images de chiffres manuscrits (0 à 9) d'une résolution de 28x28 pixels labellisées. C'est sur ces données que les perceptrons monocouche et multicouche ont été entraînés et testés. On sépare les données en deux, 10000 images pour le test et 60000 pour l'entraînement. Pour le cas du

perceptron monocouche il faut diviser les données en deux classes distinctes, nous choisissons 0 et 1 pour rester dans le binaire. Les images ayant comme label le chiffre qu'elle représente il suffit d'appliquer des conditions aux matrices contenant les labels ce qui nous rendra une matrice de booléen avec laquelle on peut récupérer les indices des images contenant 0 et 1. On les transforment ensuite en vecteurs de 784 éléments, cela simplifiera les calculs.

La taille de la matrice d'entrée pouvant influencer sur les résultats on diminue la résolution des images via l'algorithme du plus proche voisin, les images sont downscale vers du 14x14 pixels.

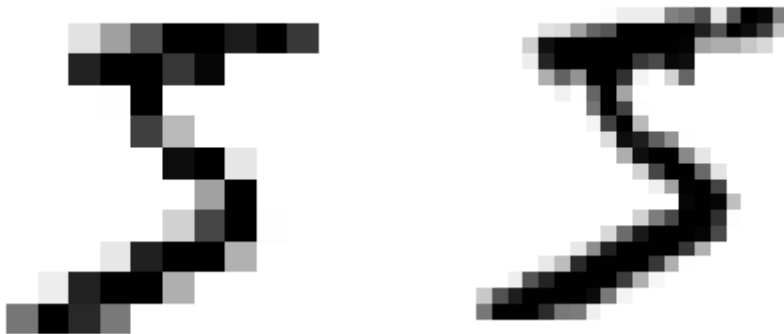


Figure 4 : Image du MNIST en 14x14 à gauche et 28x28 à droite (5)

3.2. Perceptron monocouche

Deux modèles de perceptron monocouche ont été traités, le cas classique de Rosenblatt et une variante suivant la règle de Hebb. Dans le premier cas la fonction d'activation est la fonction signum, renvoyant -1 ou 1 en fonction du signe de l'entrée dans l'autre cas c'est une fonction step, elle renvoie 0 si l'entrée est négative et 1 sinon. Ce seront les données du MNIST classées précédemment qui serviront aux tests dans les deux cas.

3.3. Perceptron multicouche

Dans ce cas, la fonction que l'on cherche à minimiser est la variance comme décrite précédemment. Le modèle ne sera doté que de trois couches : 784 entrées, 10 neurones cachés et 10 sorties. Les fonctions d'activation traitées sont ReLu et sigmoïde et afin d'avoir des résultats sous forme de probabilités la fonction softmax sera appliquée avant la sortie.

$$ReLU(x) = \max(0, x)$$

$$\text{sigmoïde}(x) = \frac{1}{1+e^{-x}}$$

$$\text{Softmax} = \frac{e^x}{\sum_{k=1}^K e^{kx}}, x \in \mathbb{R}$$

```

lr rate : 0.1
| Itération : 100/1000 | Précision : 35.00% | Temps : 17.29 s |
| Itération : 200/1000 | Précision : 49.87% | Temps : 17.39 s |
| Itération : 300/1000 | Précision : 57.67% | Temps : 17.36 s |
| Itération : 400/1000 | Précision : 62.21% | Temps : 17.18 s |
| Itération : 500/1000 | Précision : 64.65% | Temps : 17.03 s |
| Itération : 600/1000 | Précision : 66.39% | Temps : 17.14 s |
| Itération : 700/1000 | Précision : 68.06% | Temps : 17.27 s |
| Itération : 800/1000 | Précision : 68.97% | Temps : 17.16 s |
| Itération : 900/1000 | Précision : 69.59% | Temps : 17.12 s |
| Itération : 1000/1000 | Précision : 70.21% | Temps : 17.31 s |
Précision entraînement : 70.21%
Temps d'entraînement : 172.25 s
Précision test : 70.61%

```

Figure 5 : Invite de commande, exécution de l'algorithme avec sigmoïde

3.4. Entraînement

L'entraînement se faisant sur des images de 28x28 pixels (donc 784 entrées) la convergence des perceptron monocouche est très grande (plus de 30000 itérations). Les perceptron seront donc entraînés suivant un nombre d'itérations définies par l'utilisateur. Les perceptron prendront donc les 60000 images en entrée et appliqueront leur algorithme jusqu'à ce que le nombre d'itérations soit atteint sans conditions sur le nombre d'erreurs.

3.5. Evaluation des performances

L'analyse des performances se fera par le biais de la précision (le rapport du nombre de sorties correspondant au label réel sur le nombre total d'entrées) et de la variance normalisée ainsi que des matrices de confusion afin de visualiser une éventuelle faiblesse dans la classification.

4. Résultats

4.1. Perceptron monocouche

Les résultats sont grandement influencés par le learning rate, on voit que si il est trop haut le perceptron dégénère et ne rend qu'une classe par cycle sur les données puis l'autre classe dans le cycle suivant. Au dessus d'une valeur de lrate de 10^{-5} le système est résolu presque immédiatement, entre 1 et 5 itérations, et en dessous la précision évolue logarithmiquement (Figure 6a).

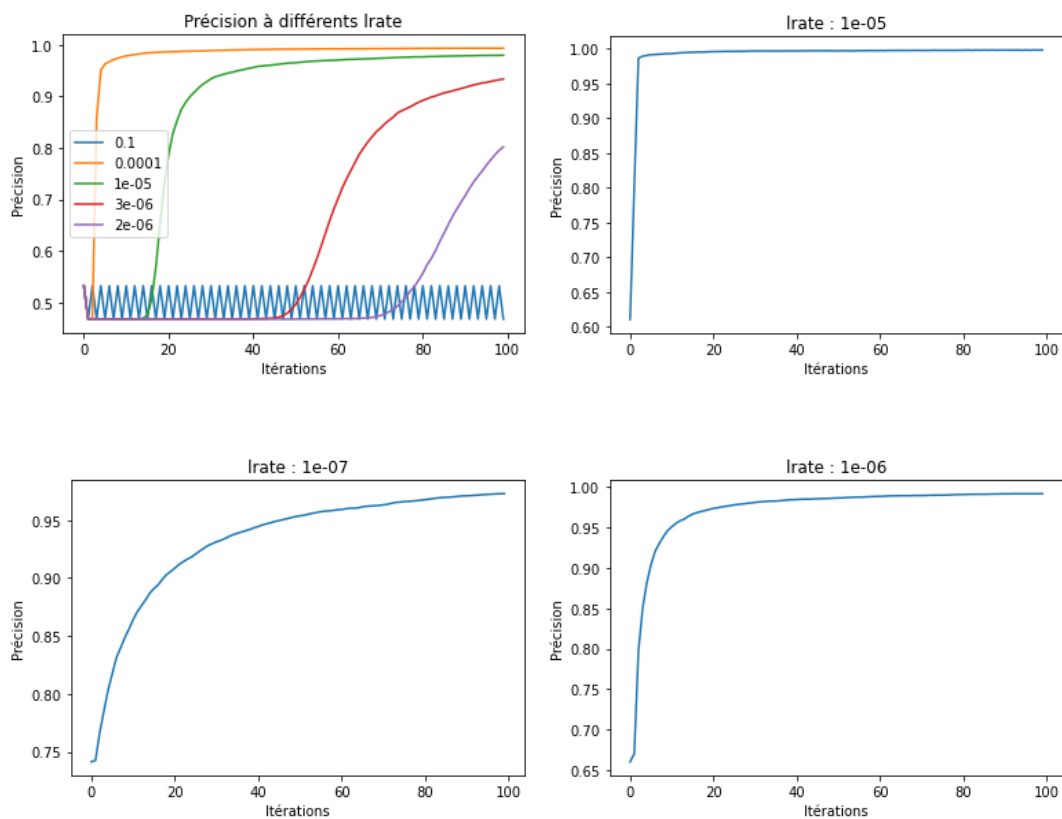


Figure 6a : Evolution de la précision d'entraînement au cours de itérations (Perceptron monocouche)

Aussi lorsque le lrate est trop bas on peut observer une évolution semblable à des escaliers sur de très petites distances et va jusqu'à s'aplatir complètement si l'on diminue encore (Figure 6b).

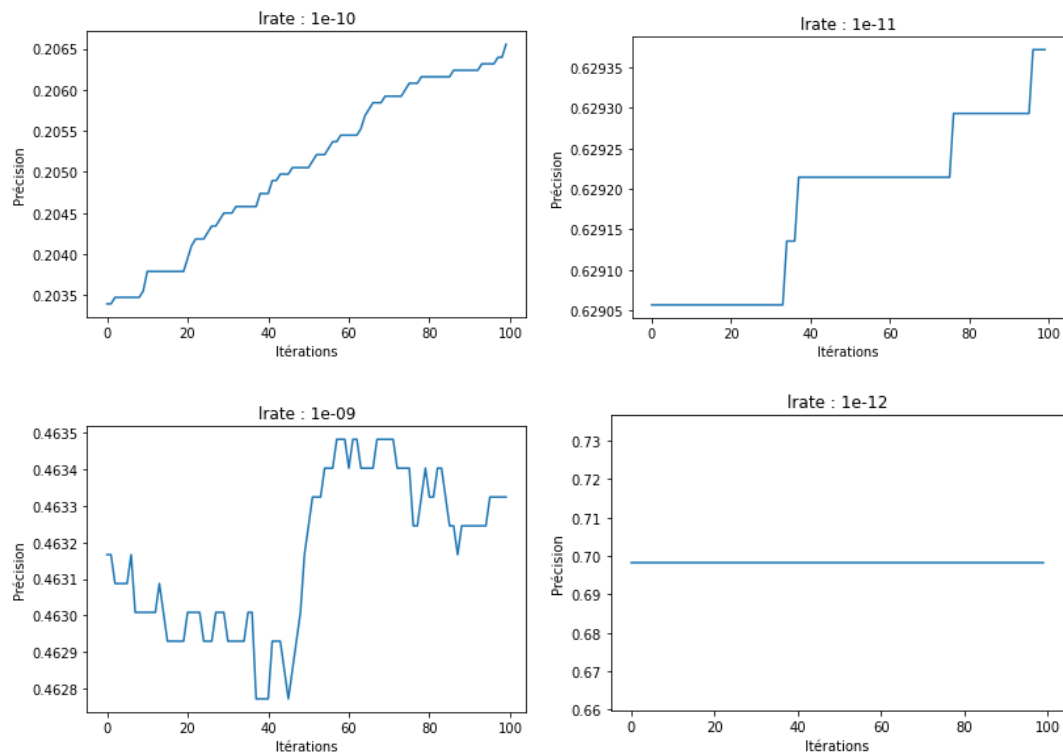


Figure 6b : Evolution de la précision d'entraînement au cours de itérations (Perceptron monocouche)

Sur 100 itérations et un lrate de 10^{-5} on peut voir que la précision c'est stabilisée, regardons les matrices de confusion et des poids.

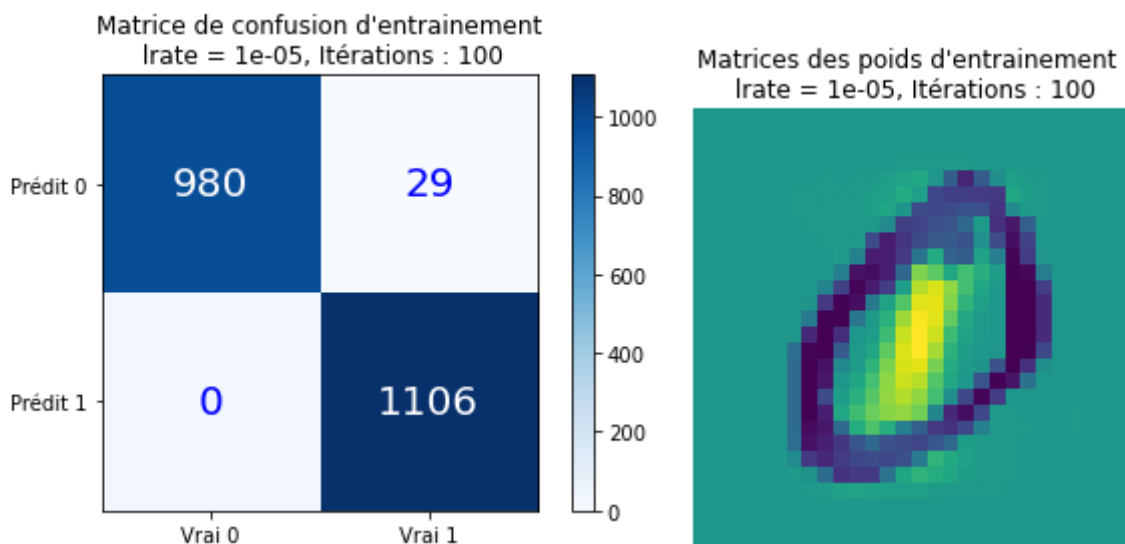


Figure 7 : Matrice de confusion (a) et Matrice des poids (b)

La quasi totalité des données (0 et 1) ont été classées correctement, il reste quelques faux négatifs mais c'est moins d'1% (Figure 7a). L'image de la matrice des poids nous montre quels poids influent le plus sur la décision, les jaunes sont les plus grandes valeurs positives, elles influent sur la décision de sortir un 1 et les valeurs bleues sont les plus grandes négatives et influent donc le plus sur la décision d'un -1 (et donc d'un 0) (Figure 7b).

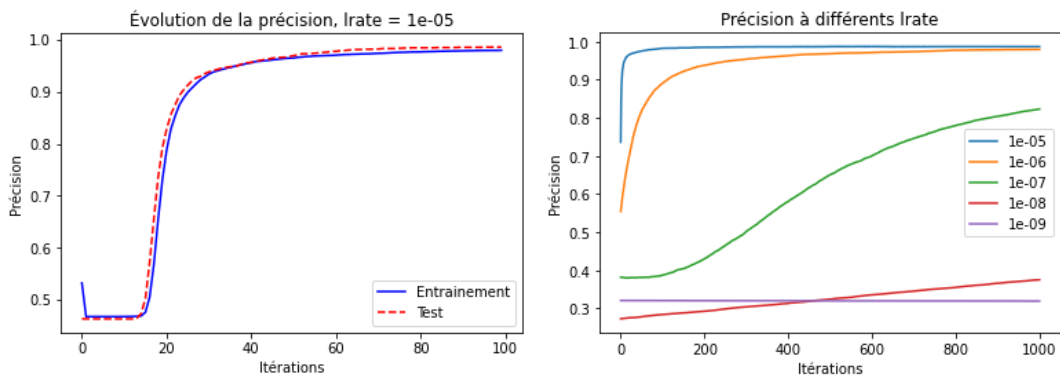


Figure 8a : Evolution des la précision de test et d'entraînement

Figure 8b : Evolution de la précision d'entraînement à différents lr

Enfin on peut regarder les performances lors du test, on voit que les deux courbes se suivent mais ne sont pas identiques, la précision sur les données de test va au dessus et en dessous de la courbe de précision sur les données d'entraînement mais sans diverger fortement l'une de l'autre le modèle ne se sûr-entraîne donc pas et les poids trouvés lors de l'entraînement s'applique parfaitement aux données de test.

Faire varier le nombre d'itérations ne revient qu'à changer l'échelle (à différents lr) (Figure 8b)

4.2. Perceptron Multicouche

On regarde maintenant les performances sur toutes les données (0 à 9).

L'évolution de la précision se fait toujours logarithmiquement et on peut voir que plus le lr diminue, plus la croissance ralentie. Aussi comparativement la croissance de la précision est beaucoup moins rapide que dans le cas binaire.

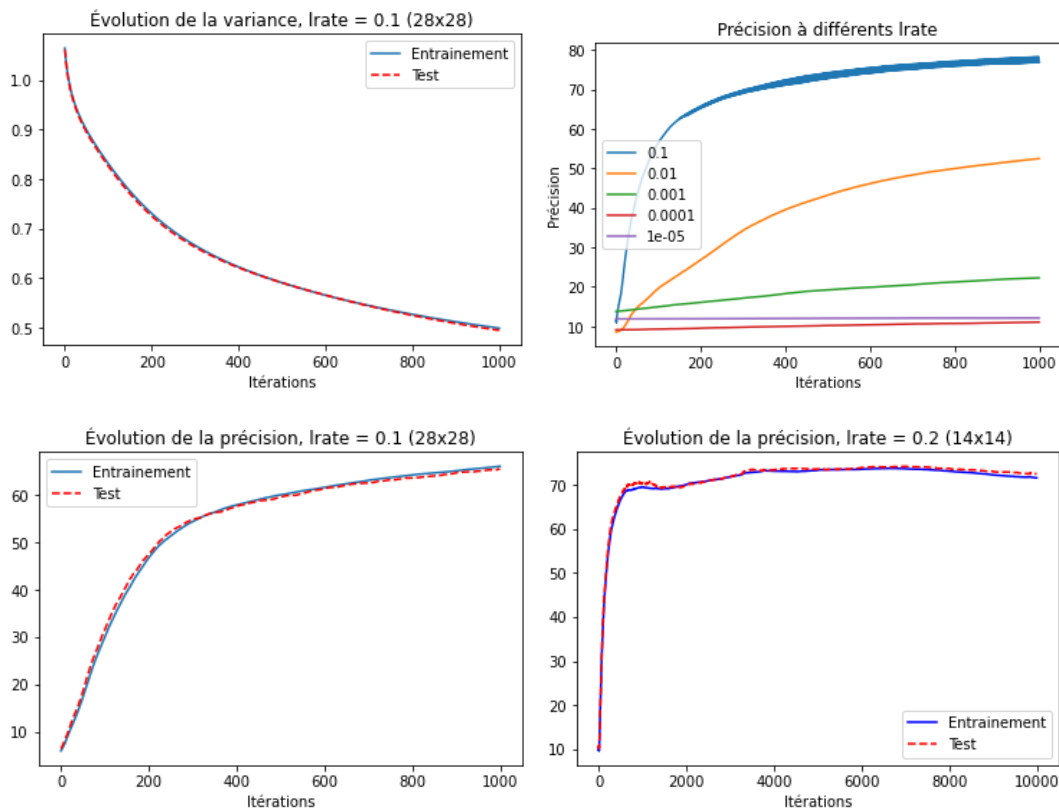


Figure 9 : (a) Evolution de la variance d'entraînement et de test ; (b) Evolution de la précision d'entraînement à différents lr rate ; (c) Evolution de la précision d'entraînement et de test ; (d)Évolution de la précision d'entraînement sur 10000 itérations

On remarque que la précision stagne autour des 70% lorsque l'on laisse le système évoluer assez longtemps, c'est peut être dû à la quantité des données ou à la grande diversité de formes pour chaque chiffre.

Prédit : 4 Réel : 3



Figure 10 : Exemple de chiffre étrange

Regardons les matrices de poids appliquées aux entrées. On voit comme pour le perceptron binaire la forme des chiffres liés aux matrices relativement clairement via la couleur des poids. Les matrices n'ont pas d'ordre et l'attribution

se fait aléatoirement lors de l'entraînement, les chiffres au-dessus de chaque matrice correspondent à la position de celle-ci dans la matrice principale.

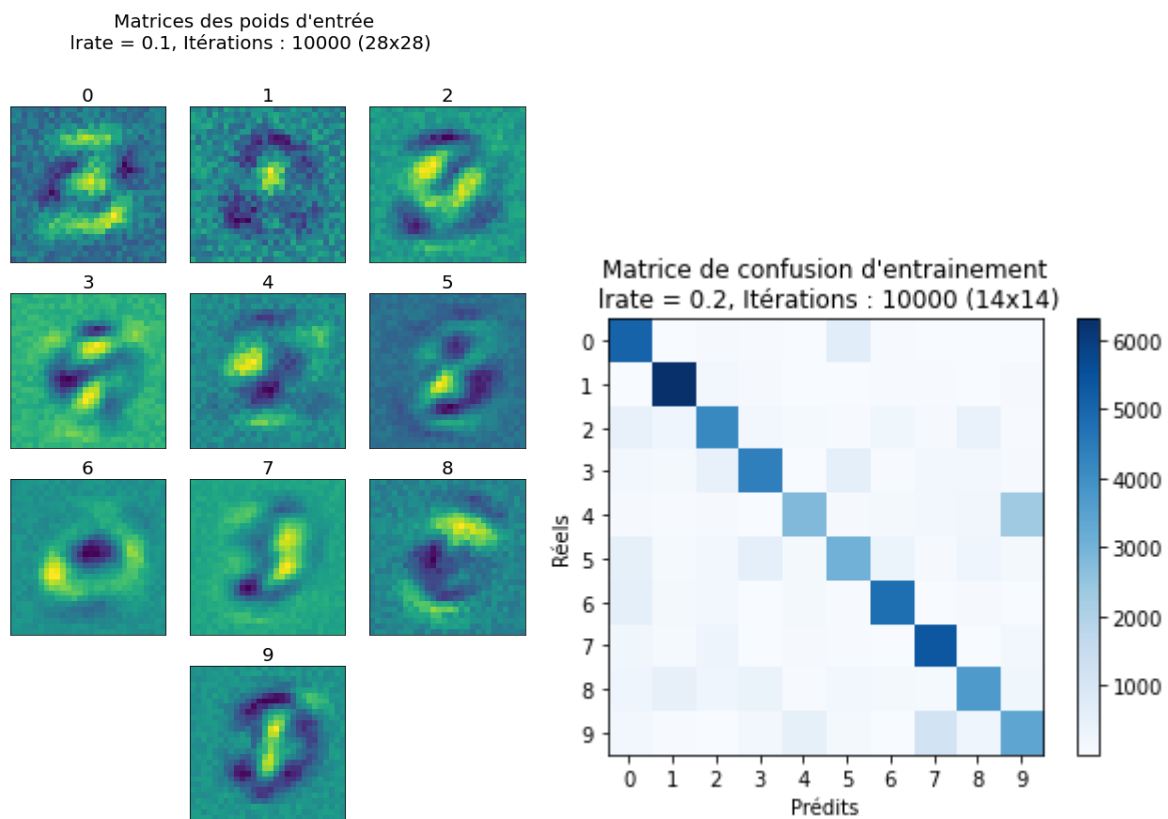


Figure 11 : Matrice des poids à gauche et matrice de confusion à droite

La matrice de confusion nous montre que pour la grande majorité les éléments sont bien classés cependant on peut voir qu'il y a encore des difficultés à différencier certains chiffres ; on voit qu'il prédit relativement souvent qu'un 4 est un 9.

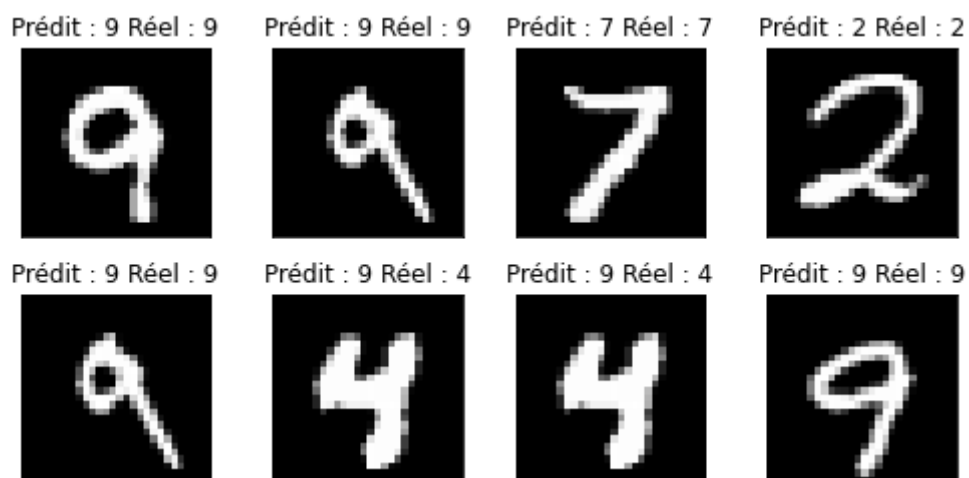


Figure 12 : image des chiffres passés dans le perceptron et la prédiction faite

5. Conclusion

Le perceptron est le plus basique des modèles de réseaux de neurones et permet de résoudre des problèmes de classification simples. L'algorithme est efficace et relativement rapide mais on voit les limites rapidement, l'algorithme peut dégénérer si une mauvaise fonction d'activation est utilisée (ReLU et les neurones morts) ou ne jamais apprendre si le pas d'apprentissage est trop grand. Aussi si les données sont trop complexes l'algorithme ne s'améliore presque plus (très lentement) au risque d'être moins performant sur des données inconnues. Afin de traiter des données plus complexes (images d'animaux et d'objets) il est indispensable d'étudier les réseaux convolutifs.

6. Références

3Blue1Brown, What is backpropagation really doing? | Chapter 3, Deep learning,
<https://www.3blue1brown.com/lessons/backpropagation-calculus>

Michael Nielsen, Neural Networks and Deep Learning (le 08/12/2022),
<http://neuralnetworksanddeeplearning.com/>

Wikipédia, Perceptron (le 08/12/2022),
https://fr.wikipedia.org/wiki/Perceptron_multicouche

Wikipédia, Méthode des k plus proches voisins
(le 08/12/2022),
https://fr.wikipedia.org/wiki/M%C3%A9thode_des_k_plus_proches_voisins